Индексы и их виды. Пользовательские функции и триггеры.

старший преподаватель кафедры АСУ
Валова Анастасия Александровна

Особенности хранения данных

Изначально данные хранятся в **неупорядоченном виде.**

\+ нет накладных расходов на поддержку индексов.

\- поиск возможен только через полное сканирование кучи.

Куча (Heap)

№п/п	LastName	City
1	Блинов	Рязань
…		
200	Григорьев	Москва
…		
350	Аристов	Тула
…		
400	Васильев	Орёл

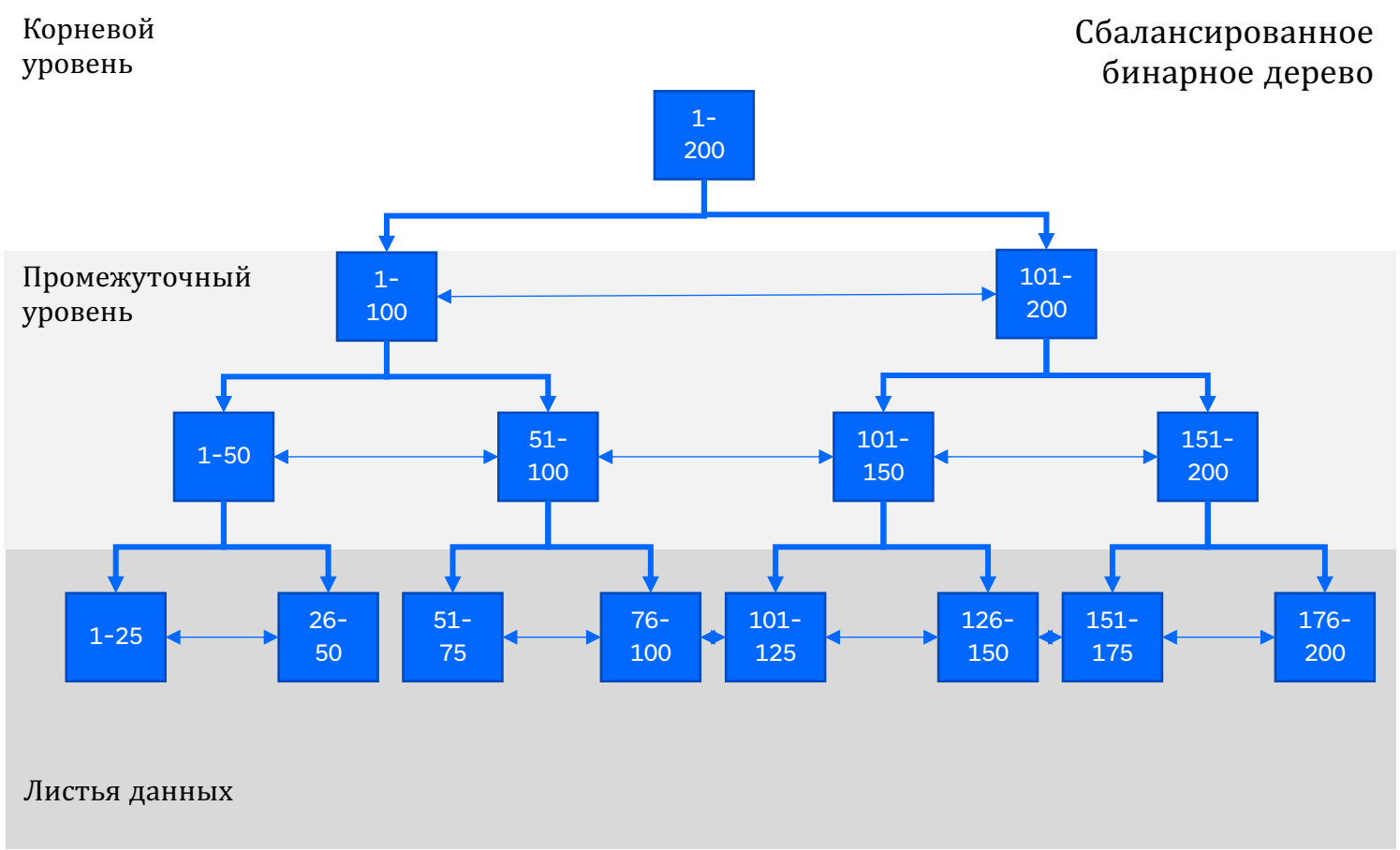
Индекс

LastName	№п/п
Аристов	350
…	
Блинов	1
…	
Васильев	400
…	
Григорьев	200

Структура и принцип работы индекса

Индекс – это структура, которая связана с таблицей или представлением и ускоряет получение строк из таблицы или представления.

- поиск данных O(logN) вместо O(N)



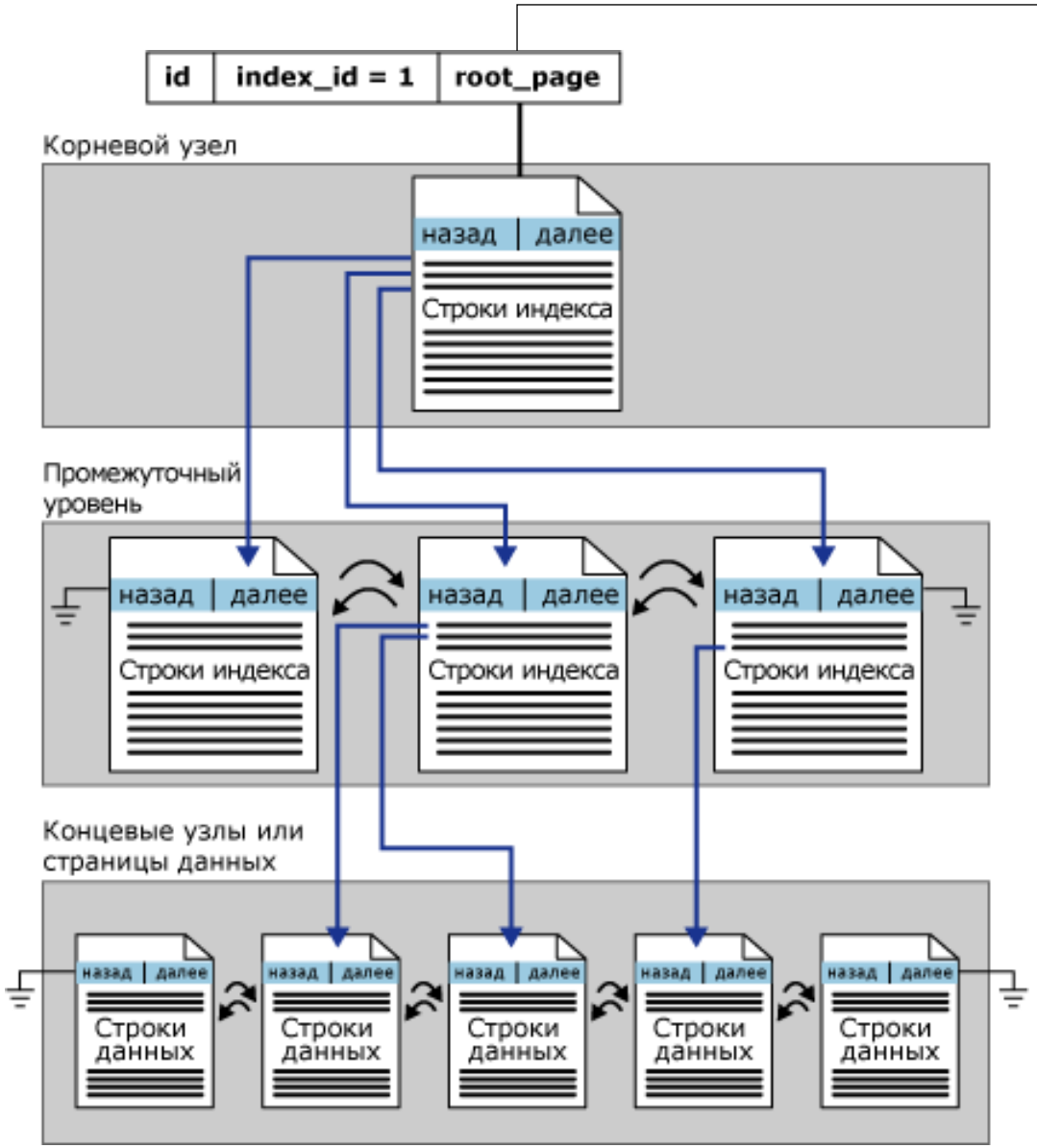
Кол-во строк	Максимум итераций
8	3
100	7
1 000	10
10 000	14
100 000	17
1 000 000	20

Виды индексов

1. Кластерный;
2. Некластерный;
3. Колоночный;
4. Полнотекстовый (Full Text);
5. XML;
6. Пространственный (Spatial).

https://docs.microsoft.com/ru-ru/sql/relational-databases/indexes/indexes?
https://docs.microsoft.com/ru-ru/sql/relational-databases/sql-server-index-design-guide?

Кластерный индекс (clustered)



Ядро СУБД используется системное представление sys.partitions для получения индекса:

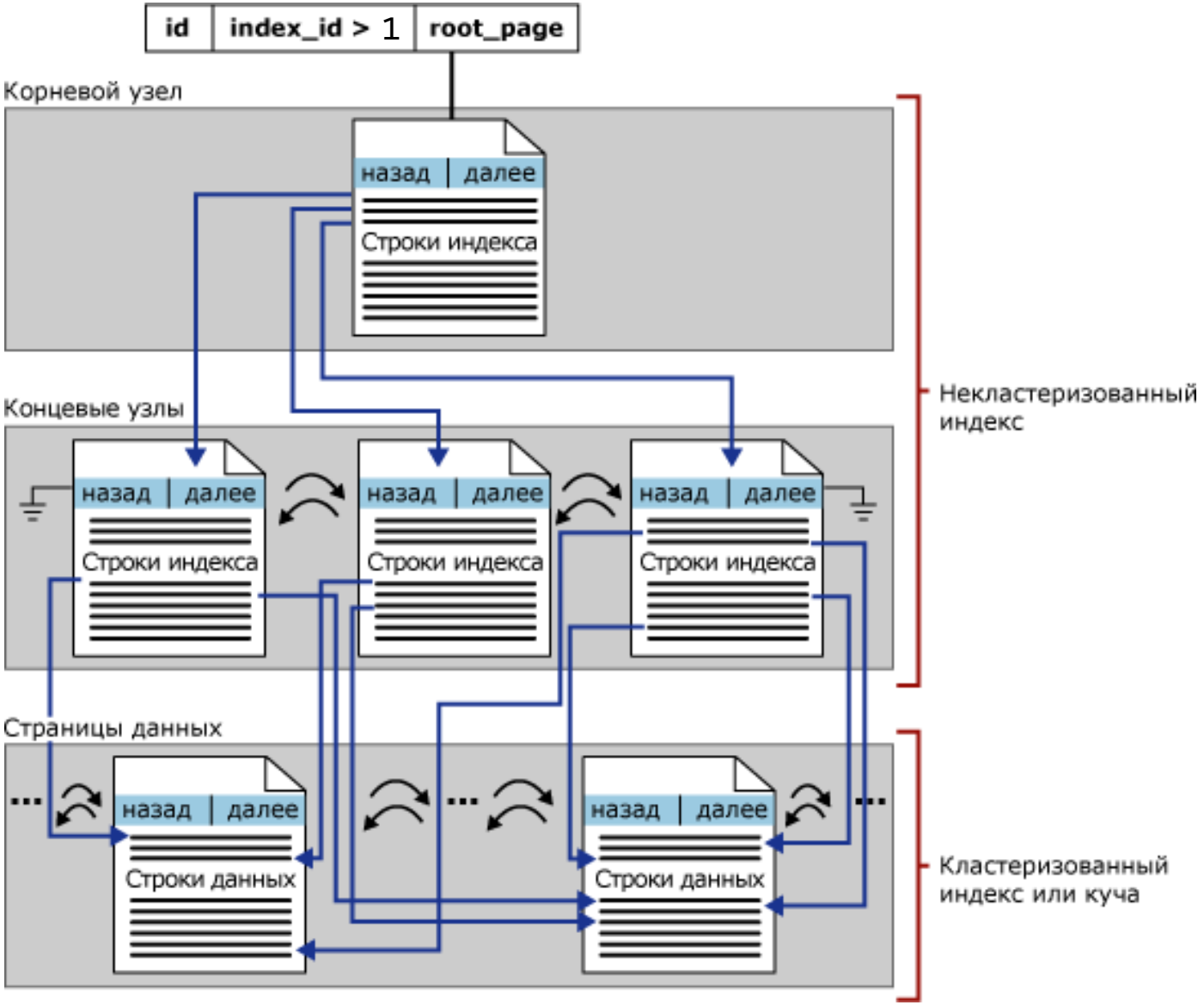
- Index_id = 1 - кластеризованный индекс;
- Index_id = 0 – куча
- Index_id = 2 и более - некластеризованный индекс;

Кластерный индекс (clustered)

- Может быть только 1 кластерный индекс на таблицу;
- Является частью таблицы;
- Хранится в той же файловой группе, где и сама таблица;
- Может содержать в себе максимум 32 столбцов
  - 31 столбцов, если в таблице имеется один или несколько индексов XML
- Для ограничения PRIMARY KEY создается уникальный кластерный индекс по умолчанию;
- Кластерный индекс должен быть как можно меньше (насколько это возможно);
- Необязательный, но рекомендуемый.

https://docs.microsoft.com/ru-ru/sql/relational-databases/indexes/create-clustered-indexes
https://docs.microsoft.com/en-us/sql/sql-server/maximum-capacity-specifications-for-sql-server

Некластерный индекс (non-clustered)




```
CREATE NONCLUSTERED INDEX IX_Users_Active
ON Users(Email)
WHERE IsActive = 1;
```

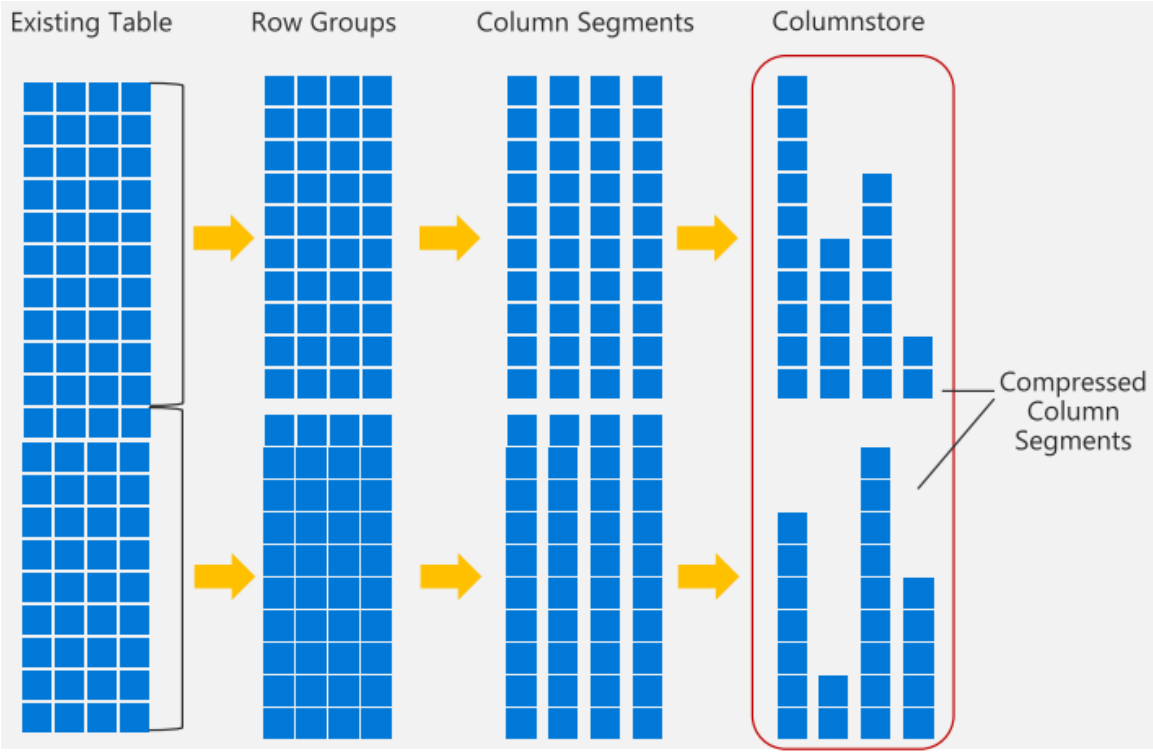
Некластерный индекс (non-clustered)

- Количество некластерных индексов у таблицы 999;
- Может содержать в себе максимум 16 столбцов;
- Некластерные индексы всегда содержат столбцы кластерного индекса (когда у таблицы есть кластерный индекс);
- Если таблица является «кучей», то каждый некластерный индекс содержит в себе идентификатором строки в таблице (RID);
- Может быть создан в любой файловой группе данной БД;
- Может быть с условием (фильтром), чтобы иметь меньший размер.

https://docs.microsoft.com/ru-ru/sql/relational-databases/indexes/create-nonclustered-indexes?

Колоночный индекс (Columnstore)

- Хранение данных в колоночном виде (в отличие от построчного в классических индексах);
- Высокий уровень сжатия данных (до 10 раз);
- Идеально подходят для хранилищ данных (DW) и мало для OLTP;



https://docs.microsoft.com/ru-ru/sql/relational-databases/indexes/columnstore-indexes-overview?

Полнотекстовый индекс (Full Text)

- Не используется при обычных запроса T-SQL;
- Для использования служат предикаты (CONTAINS, CONTAINSTABLE, FREETEXT, FREETEXTTABLE);
- Может использоваться, чтобы осуществлять поиск по документам (doc/docx, xls/xlsx, pdf), сохраненным в БД;
- Для таблицы или индексированного представления допускается только один полнотекстовый индекс;
- Полнотекстовый индекс может содержать не более 1024 столбцов;
- Может индексировать XML-документы, но только индексирует значения, не теги;
- Natural Language Search.

```
CREATE FULLTEXT INDEX ON
Products(ProductDescription) KEY INDEX PK_Products
ON FT_Catalog;

SELECT * FROM Products WHERE
CONTAINS(ProductDescription, 'энергосберегающий');
```

https://docs.microsoft.com/ru-ru/sql/relational-databases/search/create-and-manage-full-text-indexes?

XML индекс (XML index)

- Позволяет индексировать определенные узлы XML-документа;
- 249 XML индексов на одну таблицу;
- Требует кластерного индекса на таблице;
- У каждого XML столбца может быть единственный основной (Primary) индекс XML и множество вторичных индексов XML;
- XML индексы могут быть созданы на единственном XML столбце;
- Нет online перестроения;
- Не доступны для переменных XML. Только используются на таблицах.

https://docs.microsoft.com/ru-ru/sql/relational-databases/xml/xml-indexes-sql-server?

Пространственный индекс (Spatial)

- Пространственный индекс можно создать только на столбце типа **geometry** или **geography**;
- Пространственные индексы можно определить только для таблицы, у которой имеется первичный ключ;
- Можно создать до 249 пространственных индексов для каждого пространственного столбца в таблице;
- Для построения индексов нельзя воспользоваться доступным параллелизмом процессов;
- Пока для таблицы определен пространственный индекс, изменить метаданные первичного ключа невозможно;
- Пространственные индексы не могут быть определены для индексированных представлений.

https://docs.microsoft.com/ru-ru/sql/relational-databases/spatial/spatial-indexes-overview?

Индексированные представления

Это представление, которое имеет один или более индексов.

Преимущества:

- Агрегированные данные могут вычисляться и входить в состав индекса, чтобы избегать дорогостоящих вычислений при выполнении запроса;
- Таблицы могут быть соединены, а результат – сохранен в индексе.

https://docs.microsoft.com/ru-ru/sql/relational-databases/views/create-indexed-views?

13 Триггеры

Триггеры и типы триггеров

Триггер – это хранимая процедура, автоматически выполняемая при наступлении некоторого события.

В SQL Server Trigger есть следующие типы триггеров:

- **DML**
  - INSERT, UPDATE и DELETE
- **DDL**
  - При создании / удалении объектов
- **Logon**
  - При входе пользователя

DML-триггеры

- **For = After (после)**

  - «стандартный» триггер – работает в конце транзакции (дополнительно к операции изменения данных)

- **Instead Of (вместо)**

  - Работает вместо команды изменения данных

```
CREATE TRIGGER [ имя_схемы . ] имя_триггера
ON { таблица | представление }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER | INSTEAD OF }
{ [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS { операторы_sql  [ ; ] [ ,...n ]
        | EXTERNAL NAME <method specifier [ ; ] > }
```

https://docs.microsoft.com/ru-ru/sql/relational-databases/triggers/dml-triggers?

Определение изменившихся данных в DML-триггере

- **UPDATE()**

Возвращает логическое значение, указывающее, была ли сделана попытка INSERT или UPDATE в указанном столбце таблицы или представления.

```
CREATE TRIGGER reminder
ON SalesLT.Product
AFTER UPDATE
AS
IF ( UPDATE (ProductNumber) OR UPDATE (rowguid) )
BEGIN
RAISERROR (50009, 16, 10)
END;
```

- **COLUMNS_UPDATED()**

Возвращает битовую маску (varbinary), указывающую на столбцы в таблице или представлении, которые были обновлены.

```
CREATE TRIGGER dbo.updEmployeeData
ON dbo.employeeData
AFTER UPDATE AS
/* Проверяем были ли столбцы 2, 3 или 4 обновлены.*/
  IF (COLUMNS_UPDATED() & 14) > 0
            ...
```

https://docs.microsoft.com/ru-ru/sql/t-sql/functions/columns-updated-transact-sql?

Соглашения в DML-триггерах

- Таблица “**inserted**” (используется в Insert и Update);
- Таблица “**deleted**” (используется в Delete и Update);
- After-триггер участвует в транзакции:
  - Может откатить изменения в данных (при необходимости);
- Пользователю нужны права доступа к объектам, использующимся в триггере;
- Можно задавать порядок выполнения триггеров через sp_settriggerorder (@order = 'first' | 'last' | 'none’).

```
EXEC sp_settriggerorder @triggername =
'myLovelyTrigger', @order = 'first', @stmttype='update'
```

https://docs.microsoft.com/ru-ru/sql/relational-databases/triggers/use-the-inserted-and-deleted-tables?

DDL-триггеры

DDL-триггеры срабатывают **после** выполнения DDL-операторов, повлекших запуск триггера.

Используйте DDL-триггеры, если хотите сделать следующее:

- предотвращать внесение определенных изменений в схему базы данных;
- настроить выполнение в базе данных некоторых действий в ответ на изменения в схеме БД;
- записывать изменения или события схемы базы данных.

```
CREATE TRIGGER safety
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
   PRINT ‘Отключите триггер "safety" для изм./удаления таблиц!'
   ROLLBACK;
```

**DDL-триггеры нельзя использовать в качестве INSTEAD OF-триггеров.*

https://docs.microsoft.com/ru-ru/sql/relational-databases/triggers/ddl-triggers?

Триггеры входа

Триггеры входа вызывают срабатывание хранимых процедур в ответ на событие LOGON (при начале пользовательской сессии). Триггеры входа срабатывают после завершения этапа проверки подлинности при входе, но перед тем, как пользовательский сеанс реально устанавливается. Если проверка подлинности завершается сбоем, триггеры входа не срабатывают.

```
CREATE TRIGGER connection_limit_trigger
ON ALL SERVER WITH EXECUTE AS 'login_test'
FOR LOGON
AS
BEGIN
IF ORIGINAL_LOGIN()= 'login_test' AND
   (SELECT COUNT(*) FROM
            sys.dm_exec_sessions
            WHERE is_user_process = 1 AND
             original_login_name = 'login_test') > 3
   ROLLBACK;
END;
```

https://docs.microsoft.com/ru-ru/sql/relational-databases/triggers/logon-triggers?

Вложенные (каскадные) триггеры

- Вложенный триггер – это триггер, выполняемый из-за действий в другом триггере (вложенность до 32 уровней);
- Можно разрешать или запрещать вложенность (каскадность) триггеров AFTER с помощью параметра конфигурации сервера **nested triggers:** 1 – разрешает (by def.), 0 – запрещает);
- **@@NESTLEVEL** – текущий уровень вложенности хр.проц. (0, 1, …)

```
EXEC sp_configure 'show advanced options', 1;
GO
RECONFIGURE ;
GO
EXEC sp_configure 'nested triggers', 0 ;
GO
RECONFIGURE;
```

https://docs.microsoft.com/ru-ru/sql/relational-databases/triggers/create-nested-triggers?

Рекурсивные триггеры

- **Прямая рекурсия –** триггер выполняет действие, вызывающее повторное срабатывание того же триггера. По умолчанию включена, управляется параметром 'server trigger recursion’.
- **Косвенная рекурсия –** триггер срабатывает и выполняет действие, которое вызывает срабатывание другого триггера того же типа (AFTER или INSTEAD OF). Отключается через **nested triggers = 0**.

```
EXEC sp_configure 'show advanced options', 1;
GO
RECONFIGURE ;
GO
EXEC sp_configure 'server trigger recursion', 0 ;
GO
RECONFIGURE;
```

14 Пользовательские типы. Функции.

Пользовательские типы (UDT)

Вводятся при недостаточной выразительности встроенных в СУБД стандартных типах.

Виды:

- Псевдоним – на базе стандартного типа;
- Определяемый пользователем тип (.Net CLR);
- Определяемый пользователем табличный тип.

https://docs.microsoft.com/ru-ru/sql/relational-databases/clr-integration-database-objects-user-defined-types/clr-user-defined-types?

Пользовательские типы–псевдонимы

```
-- создаем тип для номера телефона
CREATE TYPE [dbo].[Phone] FROM [nvarchar](25) NULL

-- определение таблицы
CREATE TABLE [SalesLT].[Customer](
[CustomerID] [int] IDENTITY(1, 1) NOT NULL,
[Phone] [dbo].[Phone] NULL,
...
)
```

https://docs.microsoft.com/ru-ru/sql/t-sql/statements/create-type-transact-sql?

Пользовательские табличные типы

```
-- создаем табличный тип
CREATE TYPE ErrMsgTableType AS TABLE (ErrorNumber INT, ErrorMessage nvarchar(4000));
GO
-- создаем хранимую процедуру для получения данных из табличной переменной
CREATE PROCEDURE dbo.usp_LogSomeErrors
@TVP ErrMsgTableType READONLY
AS
INSERT INTO dbo.ErrorLog
(ErrorTime, UserName, ErrorNumber, ErrorMessage)
SELECT GETDATE(), CURRENT_USER, ErrorNumber, ErrorMessage FROM @TVP;
GO
-- создаем табличную переменную
DECLARE @err AS ErrMsgTableType;
-- заполняем ее данными
INSERT INTO @err (ErrorNumber, ErrorMessage)
   SELECT 0,'Ошибка0' UNION ALL
SELECT 1,'Ошибка1' UNION ALL
SELECT 2,'Ошибка2';
-- передаем в хранимую процедуру
EXEC dbo.usp_LogSomeErrors @err;
-- проверяем, что получилось в логе
select top 5 * from dbo.ErrorLog ORDER BY ErrorTime desc
```

Типы пользовательских функций (UDF)

Три типа функций:

- **Скалярные**;
  - Возвращают одно значение;
  - Вычисляются для каждой строки, если используются в SELECT;
- **Встраиваемые (inline) табличные**;
  - Возвращают переменную табличного типа;
  - Одиночный оператор SELECT определяет возвращаемую таблицу;
- **Многооператорные (multi-statement) табличные**;
  - Используется несколько операторов для формирования возвращаемой таблицы.

https://docs.microsoft.com/ru-ru/sql/relational-databases/user-defined-functions/user-defined-functions?

Ограничения пользовательских функций (UDF)

- не могут выполнять действия, изменяющие состояние БД;
- не могут содержать предложение OUTPUT INTO <таблица>;
- не могут возвращать несколько результирующих наборов;
- нет поддержки TRY...CATCH, @@ERROR и RAISERROR;
- нельзя вызывать хранимую процедуру;
- нельзя использовать динамический SQL и временные таблицы;
- нельзя использовать SET <option> и FOR XML;
- вложенность UDF не может превышать 32 уровня.

https://docs.microsoft.com/ru-ru/sql/relational-databases/user-defined-functions/create-user-defined-functions-database-engine?

Использование пользовательских функций

- помогают упростить выбираемые значения в SELECT;
- улучшают надежность и читаемость кода уменьшая количество соединений и инкапсулируя запросы;
- могут использоваться для получения агрегированных значений.

Пример скалярной функции:

```
CREATE FUNCTION [dbo].[ufnGetOrdersCount](@CustomerID int)
RETURNS int
AS
BEGIN
        DECLARE @ret int
        select @ret=count(SalesOrderID) from SalesLT.SalesOrderHeader
        WHERE CustomerID=@CustomerID
        RETURN @ret
END
GO

-- примеры использования
print [dbo].[ufnGetOrdersCount](1) -- 0
select [dbo].[ufnGetOrdersCount](30113)as [Count] -- 1
```

Пример встраиваемой табличной функции:

```
CREATE FUNCTION [dbo].[ufnGetOrderedProducts](@CustomerID int)
RETURNS TABLE
AS                    -- Возвращаем все заказанные клиентом товары
RETURN (
   SELECT c.CustomerID, c.FirstName, c.LastName,
          h.SalesOrderID,h.SalesOrderNumber,
          p.[Name] as ProductName, p.ListPrice,
          d.UnitPrice as OrderedPrice,
          [dbo].[ufnGetOrdersCount](c.CustomerID) as OrdersCount
   FROM SalesLT.SalesOrderDetail as d
   INNER JOIN SalesLT.Product as p ON p.ProductID=d.ProductID
   INNER JOIN SalesLT.SalesOrderHeader as h ON d.SalesOrderID=h.SalesOrderID
   INNER JOIN SalesLT.Customer as c ON c.CustomerID=h.CustomerID
   WHERE h.CustomerID = @CustomerID        -- ORDER BY => ERROR
)
GO

-- пример использования табличной функции
select * from [dbo].[ufnGetOrderedProducts](30113)
```

Пример многооператорной табличной функции:

```
CREATE FUNCTION [dbo].[ufnGetAllCategories]()
RETURNS @retCategoryInformation TABLE (    -- Структура возвращаемой таблицы
[ParentProductCategoryName] [nvarchar](50) NULL,
[ProductCategoryName] [nvarchar](50) NOT NULL,  [ProductCategoryID] [int] NOT NULL) AS
BEGIN
  WITH CategoryCTE([ParentProductCategoryID], [ProductCategoryID], [Name]) AS (
   SELECT [ParentProductCategoryID], [ProductCategoryID], [Name]
            FROM SalesLT.ProductCategory WHERE ParentProductCategoryID IS NULL
   UNION ALL
            SELECT C.[ParentProductCategoryID], C.[ProductCategoryID], C.[Name]
            FROM SalesLT.ProductCategory AS C
            INNER JOIN CategoryCTE AS BC ON BC.ProductCategoryID = C.ParentProductCategoryID
)
INSERT INTO @retCategoryInformation
SELECT PC.[Name] AS [ParentProductCategoryName], CCTE.[Name] as [ProductCategoryName],
CCTE.[ProductCategoryID]
FROM CategoryCTE AS CCTE INNER JOIN SalesLT.ProductCategory AS PC ON
PC.[ProductCategoryID]=CCTE.[ParentProductCategoryID];
RETURN;
END;
-- пример использования
select * FROM [dbo].[ufnGetAllCategories]()
```